

**Министерство науки и высшего образования Российской
Федерации**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ**

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Курсовая работа

**Тема: Разработка Android-приложения с
использованием архитектуры MVVM, REST API,
локального хранилища Room и фоновой
синхронизации**

Выполнил:

Чернышев Михаил Павлович

Проверил:

Шатинский Григорий Сергеевич

Санкт-Петербург
2025 г.

Содержание

1	Введение	2
2	Описание предметной области	3
3	Архитектура приложения	4
3.1	Общая структура проекта	4
3.2	Диаграмма архитектуры	4
3.3	Паттерн Repository	5
4	Проектирование навигации	6
4.1	Навигационная модель приложения	6
4.2	Навигационный граф	6
4.3	Интеграция с BottomNavigationView	6
5	Структура базы данных	8
5.1	Entity Message	8
5.2	Data Access Object (DAO)	8
6	Реализация пользовательского интерфейса	10
6.1	Кастомный элемент списка сообщений	10
6.2	Material Design компоненты	11
6.3	Реализация функционала «лайков»	11
7	Фоновая синхронизация данных	13
7.1	Настройка WorkManager	13
7.2	Реализация SyncWorker	14
8	Система уведомлений	15
8.1	Создание канала уведомлений	15
8.2	Отображение уведомлений	15
9	Запрос разрешений	17
10	Обработка состояния сети	18
11	Заключение	19
11.1	Достигнутые результаты	19
11.2	Использованные технологии и инструменты	19
11.3	Практическая значимость	20
11.4	Возможные направления развития	20

1 Введение

Целью данной курсовой работы является разработка Android-приложения с улучшенным пользовательским интерфейсом, поддержкой фоновой синхронизации данных и системой уведомлений.

В рамках работы были реализованы следующие задачи:

1. Создание кастомного элемента списка сообщений с отображением аватарки, имени автора и текста;
2. Применение компонентов Material Design: FloatingActionButton, AppBarLayout, MaterialCardView;
3. Реализация функционала «лайков» сообщений с изменением иконки при клике;
4. Настройка WorkManager для периодической фоновой синхронизации данных из сети;
5. Отображение системных уведомлений при успешной синхронизации;
6. Реализация запроса разрешений при первом запуске приложения;
7. Обработка изменений состояния сети (онлайн/офлайн).

Приложение разработано с использованием архитектурного паттерна MVVM, обеспечивающего разделение ответственности между компонентами и устойчивость к изменениям конфигурации устройства.

2 Описание предметной области

Разрабатываемое приложение представляет собой мобильный клиент для просмотра новостной ленты. Приложение загружает данные из внешнего REST API (Reddit API), отображает их в удобном для пользователя виде и обеспечивает возможность взаимодействия с контентом.

Основные сценарии использования приложения:

- Просмотр ленты новостей с возможностью прокрутки;
- Обновление данных по запросу пользователя (pull-to-refresh);
- Отметка понравившихся сообщений (функционал «лайков»);
- Работа в офлайн-режиме с использованием кэшированных данных;
- Получение уведомлений о новых данных.

3 Архитектура приложения

3.1 Общая структура проекта

Проект организован в соответствии с архитектурой MVVM и состоит из следующих пакетов:

- data/ — сущности базы данных Room (Message, MessageDao, AppDatabase);
- repository/ — репозиторий для централизованного доступа к данным;
- network/ — сетевой слой (Retrofit, API интерфейсы);
- ui/ — фрагменты и ViewModel для каждого экрана;
- worker/ — фоновые задачи WorkManager;
- receiver/ — BroadcastReceiver для отслеживания сети;
- util/ — вспомогательные утилиты.

3.2 Диаграмма архитектуры

На рисунке 1 представлена общая архитектура приложения.

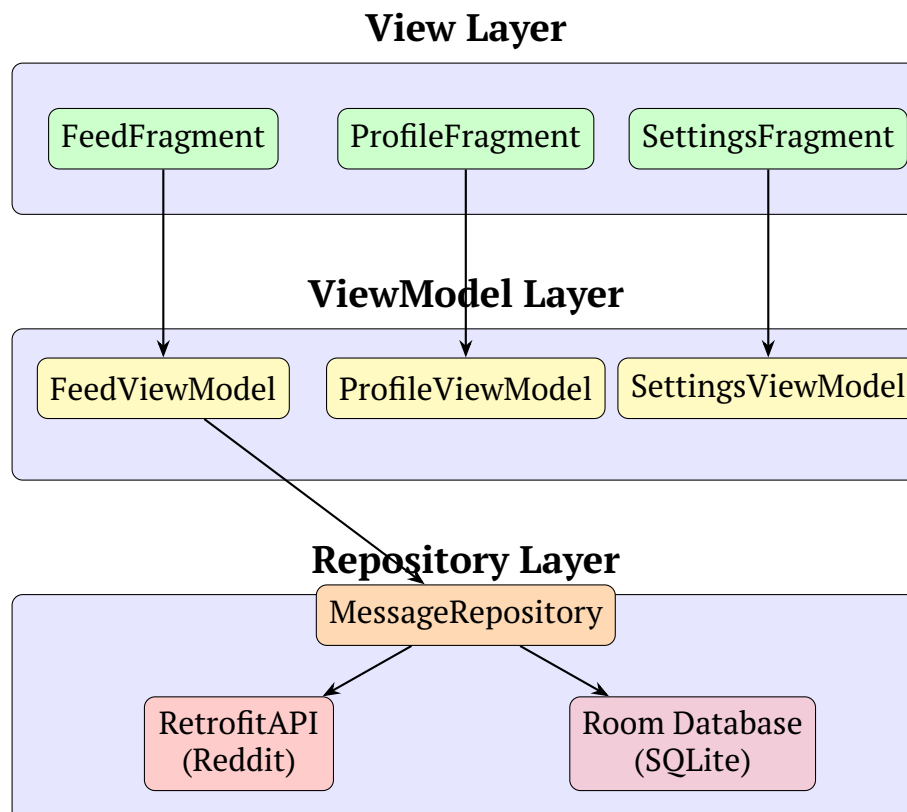


Рис. 1: Архитектура приложения MVVM

3.3 Паттерн Repository

Паттерн Repository обеспечивает единую точку доступа к данным, абстрагируя источники данных от остальной части приложения. MessageRepository реализует стратегию «offline-first»:

1. При запросе данных сначала выполняется попытка загрузки из сети;
2. Полученные данные сохраняются в локальную базу данных;
3. При отсутствии сети возвращаются данные из локального кэша;
4. UI подписывается на Flow из базы данных и автоматически обновляется.

Такой подход обеспечивает:

- Работоспособность приложения в офлайн-режиме;
- Мгновенное отображение кэшированных данных при запуске;
- Автоматическое обновление UI при изменении данных в БД.

4 Проектирование навигации

4.1 Навигационная модель приложения

Навигация в приложении реализована с использованием компонента Navigation из Android Jetpack. Приложение содержит три основных экрана, доступных через нижнюю панель навигации:

1. **Лента сообщений (FeedFragment)** — главный экран с новостной лентой;
2. **Профиль (ProfileFragment)** — информация о пользователе;
3. **Настройки (SettingsFragment)** — параметры приложения.

4.2 Навигационный граф

Навигационный граф определяется в файле `nav_graph.xml` и содержит описание всех экранов (destinations) и переходов между ними.

```
1 <navigation
2   xmlns:android="http://schemas.android.com/apk/res/android"
3   app:startDestination="@id/feedFragment">
4
5   <fragment
6     android:id="@+id/feedFragment"
7     android:name="com.example.messenger.ui.feed.FeedFragment"
8     android:label="Feed" />
9
10  <fragment
11    android:id="@+id/profileFragment"
12    android:name="com.example.messenger.ui.profile.
13      ProfileFragment"
14    android:label="Profile" />
15
16  <fragment
17    android:id="@+id/settingsFragment"
18    android:name="com.example.messenger.ui.settings.
19      SettingsFragment"
20    android:label="Settings" />
21 </navigation>
```

Листинг 1: Навигационный граф `nav_graph.xml`

4.3 Интеграция с BottomNavigationView

Нижняя панель навигации связывается с `NavController` для автоматического переключения между экранами:

```
1 val navHostFragment = supportFragmentManager
2   .findFragmentById(R.id.nav_host_fragment) as NavHostFragment
3 val navController = navHostFragment.navController
4
5 val bottomNav = findViewById<BottomNavigationView>(R.id.
6   bottom_navigation)
7 bottomNav.setupWithNavController(navController)
```

Листинг 2: Setup navigation in MainActivity

5 Структура базы данных

5.1 Entity Message

Для хранения сообщений используется таблица `messages`, описываемая Entity-классом `Message`:

```
1 @Entity(tableName = "messages")
2 data class Message(
3     @PrimaryKey val id: String,
4     val title: String,
5     val body: String,
6     val author: String,
7     val avatarUrl: String = "",
8     val isLiked: Boolean = false,
9     val timestamp: Long = System.currentTimeMillis()
10 )
```

Листинг 3: Entity Message

Поля таблицы:

- `id` — уникальный идентификатор сообщения (первичный ключ);
- `title` — заголовок сообщения;
- `body` — текст сообщения или URL;
- `author` — имя автора;
- `avatarUrl` — URL аватарки автора;
- `isLiked` — флаг «лайка» (сохраняется локально);
- `timestamp` — время добавления записи.

5.2 Data Access Object (DAO)

`MessageDao` определяет операции для работы с таблицей сообщений:

```
1 @Dao
2 interface MessageDao {
3     @Query("SELECT * FROM messages ORDER BY timestamp DESC")
4     fun getAllMessagesFlow(): Flow<List<Message>>
5
6     @Query("SELECT * FROM messages ORDER BY timestamp DESC")
7     suspend fun getAllMessages(): List<Message>
8
9     @Insert(onConflict = OnConflictStrategy.REPLACE)
10    suspend fun insertAll(messages: List<Message>)
11 }
```

```
12 @Query("UPDATE messages SET isLiked = :isLiked WHERE id = :id")
13 suspend fun updateLikeStatus(id: String, isLiked: Boolean)
14 }
```

Листинг 4: MessageDao interface

Особенности реализации:

- `getAllMessagesFlow()` возвращает `Flow` для реактивного обновления UI;
- `OnConflictStrategy.REPLACE` обеспечивает обновление существующих записей;
- Все операции помечены как `suspend` для использования с корутинами.

6 Реализация пользовательского интерфейса

6.1 Кастомный элемент списка сообщений

Для отображения сообщений в ленте был создан кастомный layout `item_message.xml`, включающий:

- Круглую аватарку пользователя (`ShapeableImageView`);
- Имя автора сообщения;
- Заголовок и текст сообщения;
- Кнопку «лайка» с изменяемой иконкой.

```
1 <com.google.android.material.card.MaterialCardView
2     android:layout_width="match_parent"
3     android:layout_height="wrap_content"
4     app:cardCornerRadius="12dp"
5     app:cardElevation="2dp">
6
7     <LinearLayout
8         android:orientation="horizontal"
9         android:padding="12dp">
10
11         <!-- Avatar -->
12         <com.google.android.material.imageview.ShapeableImageView
13             android:id="@+id/message_avatar"
14             android:layout_width="48dp"
15             android:layout_height="48dp"
16             app:shapeAppearanceOverlay="@style/CircleImageView" />
17
18         <!-- Content -->
19         <LinearLayout android:orientation="vertical">
20             <TextView android:id="@+id/message_author" />
21             <TextView android:id="@+id/message_title" />
22             <TextView android:id="@+id/message_body" />
23         </LinearLayout>
24
25         <!-- Like button -->
26         <ImageButton
27             android:id="@+id/message_like_button"
28             android:src="@drawable/ic_like_outline" />
29
30     </LinearLayout>
31 </com.google.android.material.card.MaterialCardView>
```

Листинг 5: Фрагмент `item_message.xml`

6.2 Material Design компоненты

В приложении используются следующие компоненты Material Design:

- **AppBarLayout** с **MaterialToolbar** — верхняя панель приложения;
- **FloatingActionButton** — кнопка быстрого обновления ленты;
- **MaterialCardView** — карточки для отображения сообщений;
- **BottomNavigationView** — нижняя навигация между экранами;
- **SwipeRefreshLayout** — обновление списка свайпом вниз.

```
1 <androidx.coordinatorlayout.widget.CoordinatorLayout>
2
3     <com.google.android.material.appbar.AppBarLayout>
4         <com.google.android.material.appbar.MaterialToolbar
5             app:layout_scrollFlags="scroll|enterAlways|snap"
6             app:title="@string/app_name" />
7     </com.google.android.material.appbar.AppBarLayout>
8
9     <androidx.fragment.app.FragmentContainerView
10         android:id="@+id/nav_host_fragment"
11         app:layout_behavior=
12             "@string/appbar_scrolling_view_behavior" />
13
14     <com.google.android.material.bottomnavigation.
15         BottomNavigationView
16         android:id="@+id/bottom_navigation"
17         android:layout_gravity="bottom" />
18 </androidx.coordinatorlayout.widget.CoordinatorLayout>
```

Листинг 6: Структура activity_main.xml

6.3 Реализация функционала «лайков»

Функционал лайков реализован через изменение состояния в базе данных и обновление UI через LiveData.

```
1 class MessageAdapter(
2     private val onLikeClick: (Message) -> Unit
3 ) : ListAdapter<Message, MessageViewHolder>(DIFF) {
4
5     class MessageViewHolder(itemView: View) {
6         fun bind(message: Message, onLikeClick: (Message) -> Unit)
7         {
8             // Update like button icon
9             likeButton.setImageResource(
```

```

9         if (message.isLiked)
10             R.drawable.ic_like_filled
11         else
12             R.drawable.ic_like_outline
13     )
14
15     likeButton.setOnClickListener {
16         onLikeClick(message)
17     }
18 }
19 }
20 }

```

Листинг 7: Обработка лайка в MessageAdapter

```

1 fun toggleLike(message: Message) {
2     viewModelScope.launch {
3         repository.toggleLike(message.id, !message.isLiked)
4     }
5 }

```

Листинг 8: Переключение лайка в FeedViewModel

7 Фоновая синхронизация данных

7.1 Настройка WorkManager

Для фоновой синхронизации данных используется библиотека WorkManager, которая обеспечивает выполнение задач с учётом системных ограничений и требований энергоэффективности.

При запуске приложения планируются две задачи:

1. **Немедленная синхронизация** (OneTimeWorkRequest) — выполняется сразу при старте;
2. **Периодическая синхронизация** (PeriodicWorkRequest) — выполняется каждые 15 минут.

```
1 class MessengerApp : Application() {
2
3     override fun onCreate() {
4         super.onCreate()
5         createNotificationChannel()
6         runImmediateSync()
7         schedulePeriodicSync()
8     }
9
10    private fun runImmediateSync() {
11        val constraints = Constraints.Builder()
12            .setRequiredNetworkType(NetworkType.CONNECTED)
13            .build()
14
15        val request = OneTimeWorkRequestBuilder<SyncWorker>()
16            .setConstraints(constraints)
17            .build()
18
19        WorkManager.getInstance(this).enqueueUniqueWork(
20            "immediate_sync",
21            ExistingWorkPolicy.REPLACE,
22            request
23        )
24    }
25
26    private fun schedulePeriodicSync() {
27        val request = PeriodicWorkRequestBuilder<SyncWorker>(
28            15, TimeUnit.MINUTES
29        ).setConstraints(constraints).build()
30
31        WorkManager.getInstance(this).enqueueUniquePeriodicWork(
32            "periodic_sync",
33            ExistingPeriodicWorkPolicy.KEEP,
34            request
35        )
36    }
37 }
```

```
36     }
37 }
```

Листинг 9: Планирование задач в MessengerApp

7.2 Реализация SyncWorker

Класс SyncWorker выполняет загрузку данных из сети и сохранение их в локальную базу данных.

```
1 class SyncWorker(
2     private val context: Context,
3     params: WorkerParameters
4 ) : CoroutineWorker(context, params) {
5
6     override suspend fun doWork(): Result {
7         Log.d(TAG, "SyncWorker started")
8
9         val repository = MessageRepository(context)
10        val result = repository.refreshMessages()
11
12        return if (result.isSuccess) {
13            val count = result.getOrElse(0)
14            showNotification("Sync complete",
15                           "Loaded $count messages")
16            Result.success()
17        } else {
18            if (runAttemptCount < 3) Result.retry()
19            else Result.failure()
20        }
21    }
22 }
```

Листинг 10: Реализация SyncWorker

8 Система уведомлений

8.1 Создание канала уведомлений

Начиная с Android 8.0 (API 26) для отображения уведомлений требуется создание канала уведомлений.

```
1 private fun createNotificationChannel() {
2     if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
3         val channel = NotificationChannel(
4             CHANNEL_ID,
5             "Data Sync",
6             NotificationManager.IMPORTANCE_DEFAULT
7         ).apply {
8             description = "Sync notifications"
9         }
10
11         val manager = getSystemService(NotificationManager::class.
12             java)
13         manager.createNotificationChannel(channel)
14     }
15 }
```

Листинг 11: Создание канала уведомлений

8.2 Отображение уведомлений

При успешной синхронизации пользователю показывается уведомление.

```
1 private fun showNotification(title: String, message: String) {
2     // Check permission for Android 13+
3     if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
4         if (ContextCompat.checkSelfPermission(context,
5             Manifest.permission.POST_NOTIFICATIONS)
6             != PackageManager.PERMISSION_GRANTED) {
7             return
8         }
9     }
10
11     val notification = NotificationCompat.Builder(context,
12         CHANNEL_ID)
13         .setSmallIcon(R.drawable.ic_sync)
14         .setContentTitle(title)
15         .setContentText(message)
16         .setPriority(NotificationCompat.PRIORITY_DEFAULT)
17         .setAutoCancel(true)
18         .build()
19
20     val manager = context.getSystemService(NotificationManager::
21         class.java)
```



```
20     manager.notify(NOTIFICATION_ID, notification)
21 }
```

Листинг 12: Отображение уведомления в SyncWorker

9 Запрос разрешений

При первом запуске приложения запрашиваются необходимые разрешения:

- **POST_NOTIFICATIONS** — для показа уведомлений (Android 13+);
- **READ_CONTACTS** — демонстрационное разрешение.

```
1 class MainActivity : AppCompatActivity() {
2
3     private val permissions = buildList {
4         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU)
5             {
6                 add(Manifest.permission.POST_NOTIFICATIONS)
7             }
8         add(Manifest.permission.READ_CONTACTS)
9     }.toTypedArray()
10
11     private val permissionLauncher = registerForActivityResult(
12         ActivityResultContracts.RequestMultiplePermissions()
13     ) { results ->
14         results.forEach { (permission, granted) ->
15             Log.d(TAG, "$permission: $granted")
16         }
17     }
18
19     private fun requestPermissionsIfNeeded() {
20         val toRequest = permissions.filter {
21             ContextCompat.checkSelfPermission(this, it)
22                 != PackageManager.PERMISSION_GRANTED
23         }
24
25         if (toRequest.isNotEmpty()) {
26             if (toRequest.any {
27                 shouldShowRequestPermissionRationale(it)
28             }) {
29                 showPermissionRationaleDialog(toRequest)
30             } else {
31                 permissionLauncher.launch(toRequest.toTypedArray())
32             }
33         }
34     }
```

Листинг 13: Запрос разрешений в MainActivity

10 Обработка состояния сети

Приложение отслеживает состояние сетевого подключения и отображает баннер при отсутствии интернета.

```
1 object NetworkUtils {
2     fun observeNetworkState(context: Context): Flow<Boolean> =
3         callbackFlow {
4             val manager = context.getSystemService(
5                 ConnectivityManager::class.java
6             )
7
8             val callback = object : NetworkCallback() {
9                 override fun onAvailable(network: Network) {
10                     trySend(true)
11                 }
12                 override fun onLost(network: Network) {
13                     trySend(false)
14                 }
15             }
16
17             val request = NetworkRequest.Builder()
18                 .addCapability(NET_CAPABILITY_INTERNET)
19                 .build()
20
21             manager.registerNetworkCallback(request, callback)
22
23             awaitClose {
24                 manager.unregisterNetworkCallback(callback)
25             }
26         }
27 }
```

Листинг 14: Наблюдение за состоянием сети в NetworkUtils

```
1 viewModel.isOnline.observe(viewLifecycleOwner) { isOnline ->
2     offlineBanner.visibility =
3         if (isOnline) View.GONE else View.VISIBLE
4 }
```

Листинг 15: Отображение баннера в FeedFragment

11 Заключение

В ходе выполнения курсовой работы было разработано функционально завершённое Android-приложение, демонстрирующее современные подходы к проектированию и разработке мобильных приложений.

11.1 Достигнутые результаты

В процессе работы были успешно решены все поставленные задачи:

1. Спроектирована и реализована архитектура приложения на основе паттерна MVVM, обеспечивающая чёткое разделение ответственности между компонентами;
2. Разработан пользовательский интерфейс с использованием компонентов Material Design 3, включая кастомные карточки сообщений с аватарками, нижнюю навигацию и плавающую кнопку действия;
3. Реализована интеграция с внешним REST API (Reddit) с использованием библиотеки Retrofit и корутин для асинхронных операций;
4. Организовано локальное хранилище данных на базе Room с поддержкой реактивного обновления UI через Flow;
5. Реализован функционал «лайков» сообщений с сохранением состояния в базе данных между сессиями;
6. Настроена фоновая синхронизация данных с использованием WorkManager, учитывающая ограничения энергопотребления и требующая наличия сетевого подключения;
7. Реализована система уведомлений для информирования пользователя о результатах синхронизации;
8. Обеспечена корректная работа приложения в офлайн-режиме благодаря стратегии offline-first и локальному кэшированию данных.

11.2 Используемые технологии и инструменты

- **Язык программирования:** Kotlin 2.0
- **Минимальная версия Android:** API 26 (Android 8.0 Oreo)
- **Архитектура:** MVVM (Model-View-ViewModel)

- **Асинхронность:** Kotlin Coroutines + Flow
- **Сетевой слой:** Retrofit 2.9 + OkHttp 4.12 + Gson
- **Локальная БД:** Room 2.6.1
- **Фоновые задачи:** WorkManager 2.9
- **Навигация:** Jetpack Navigation 2.7
- **UI компоненты:** Material Design 3
- **Загрузка изображений:** Coil 2.5

11.3 Практическая значимость

Разработанное приложение демонстрирует комплексный подход к созданию современных Android-приложений и может служить основой для разработки более сложных проектов. Применённые архитектурные решения обеспечивают:

- Масштабируемость — возможность добавления новых экранов и функций без существенной переработки существующего кода;
- Тестируемость — разделение слоёв позволяет легко писать модульные тесты;
- Поддерживаемость — чёткая структура проекта упрощает понимание и модификацию кода;
- Устойчивость — приложение корректно обрабатывает изменения конфигурации, сетевые ошибки и другие нештатные ситуации.

11.4 Возможные направления развития

В качестве дальнейшего развития проекта могут быть рассмотрены:

- Добавление авторизации пользователей;
- Реализация пагинации для загрузки большого количества сообщений;
- Добавление возможности создания и отправки собственных сообщений;
- Интеграция с другими социальными сетями и API;

- Внедрение Dependency Injection (Hilt/Dagger);
- Покрытие кода модульными и UI-тестами.

Таким образом, поставленная цель курсовой работы была полностью достигнута. Разработанное приложение соответствует современным стандартам Android-разработки и демонстрирует эффективное использование рекомендуемых Google инструментов и библиотек.